# 实验4 密码综合应用

| 姓名 | 年级专业 | 课程名称 | 实验时间 |
| --- | --- | --- | --- |
| 姚嫣如 | 23 网络空间安全 | 网络安全技术 | 2026.4.17 |

## 一、实验目的

1. 掌握 python 相关密码学库的基本用法
2. 理解并掌握混合加密体系的设计思想
3. 学习如何在不安全信道中通过 Diffie-Hellman 算法建立安全共享密钥
4. 掌握数字签名在报文完整性和身份鉴别中的作用
5. 掌握 RSA 的数学逻辑并实现 RSA 算法。

## 二、实验要求

1. 实现完整的加密通信过程，包括：

（1）Alice 和 Bob初始化公钥密码系统，Alice的密钥（$PK_A$,$SK_A$）,Bob的密钥 （$PK_B$,$SK_B$）；

（2）Diffie-Hellman 密钥协商，在物理机(Alice)和虚拟机(Bob)之间实现对称加密密钥 k 的协商；

（3）Alice 向 Bob发送数据 M，Alice首先使用协商密钥 k 对称加密数据 $C=E_k(M,H(M)$ 或 $HMAC_{k'}(M)$)；

（4）Alice 对 C签名得到Sig；

（5）Alice 发送（C, Sig）给Bob；

（6）Bob接收后验证解密。

2.通过大数幂模运算手动实现 RSA 算法

## 三、实验环境

- 操作系统：物理机（Windows） + 虚拟机（Ubuntu）
- 开发语言： Python 3.13
- 依赖库： cryptography

---

## 四、实验步骤

### （一）完整加密通信过程

1. **加密通信步骤**

   **① 非对称密钥初始化**

   Alice 和 BOB 各自生成 X25519（用于密钥交换）和 Ed25519（用于签名）的密钥对。

   👉 本次实验为单向的Alice发送，Bob验证解密，所以Bob生成的Ed25519并不会被使用。

   **② 握手阶段**

   通过 Socket 交换公钥 X25519 和 Ed25519 。

   **③ 计算共享密钥**

   利用 X25519 协议计算出同一个“共享密钥 k”。第三方即使截获了公钥也无法算出 k。由于原始生成的 key 过长或格式不固定，通常需要经过 **HKDF**（密钥派生函数）转换为 32 字节的 AES 密钥。

   **④ 加密发送**

   Alice 使用密钥交换得到的 k 对明文 M 进行加密，并且需要对 M 进行 Hash 或者 HMAC 运算，来保证数据的完整性和机密性。

   - H(M)：只是普通的哈希（如 SHA256），不需要密钥。
   - $HMAC_{k'}$（M）：是“带密钥的哈希”。它更安全，因为它使用了一个额外的密钥 k'。只有知道 k' 的人才能生成正确的哈希值。

   计算签名，将（密文 ，签名）打包封装发送。

   **⑤ 解密校验**

   Bob 收到后，先用Alice的公钥验证签名，通过后再解密。

2. **代码实现**

   Bob 服务端代码如下：

```python
import socket
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import x25519, ed25519
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
```

```python
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os

def bob_server():
    # 1. 初始化 Bob 的密钥系统
    # 用于密钥交换的 X25519
    bob_sk_x = x25519.X25519PrivateKey.generate()
    bob_pk_x = bob_sk_x.public_key()
    # 用于身份验证的 Ed25519
    bob_sk_ed = ed25519.Ed25519PrivateKey.generate()
    bob_pk_ed = bob_sk_ed.public_key()

    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(('0.0.0.0', 9999))
    server.listen(1)
    print("Bob 正在等待 Alice 连接...")

    conn, addr = server.accept()
    print(f"连接来自: {addr}")

    try:
        # 2. 交换公钥 (接收 Alice 的公钥)
        alice_pk_x_bytes = conn.recv(1024)
        alice_pk_ed_bytes = conn.recv(1024)

        # 发送 Bob 的公钥
        conn.send(bob_pk_x.public_bytes(serialization.Encoding.Raw, serialization.PublicFormat.Raw))
        conn.send(bob_pk_ed.public_bytes(serialization.Encoding.Raw, serialization.PublicFormat.Raw))

        # 3. Diffie-Hellman 密钥协商
        alice_pk_x = x25519.X25519PublicKey.from_public_bytes(alice_pk_x_bytes)
        shared_key = bob_sk_x.exchange(alice_pk_x)

        # 使用 HKDF 派生出两个密钥：k(用于加密) 和 k'(用于HMAC)
        k = HKDF(
            algorithm=hashes.SHA256(),
            length=64, # 直接派生 64 字节
            salt=None,
            info=b'handshake data'
        ).derive(shared_key)

        # 前 32 字节给 k，后 32 字节给 k'
        k_enc = derived_key[:32]
```

```
        k_hmac = derived_key[32:]
        print(f"协商成功！\nAES密钥 k: {k_enc.hex()[:10]}...\nHMAC密钥 k': {k_hmac.hex()[:10]}...")

        # 6. 接收并验证解密
        data = conn.recv(4096)
        # 接收格式: IV(16B) + Ciphertext(可变) + Signature(64B)
        iv = data[:16]
        sig = data[-64:]
        ciphertext = data[16:-64]

        # 验证 Alice 的签名
        alice_pk_ed = ed25519.Ed25519PublicKey.from_public_bytes(alice_pk_ed_bytes)
        try:
            alice_pk_ed.verify(sig, ciphertext)
            print(" 签名验证通过！消息确实来自 Alice 且密文未被篡改。")

            # 解密得到 (M + HMAC)
            cipher = Cipher(algorithms.AES(k_enc), modes.CFB(iv))
            decryptor = cipher.decryptor()
            decrypted_payload = decryptor.update(ciphertext) + decryptor.finalize()

            # 拆分 M 和 HMAC (HMAC-SHA256 固定 32 字节)
            M = decrypted_payload[:-32]
            received_hmac = decrypted_payload[-32:]

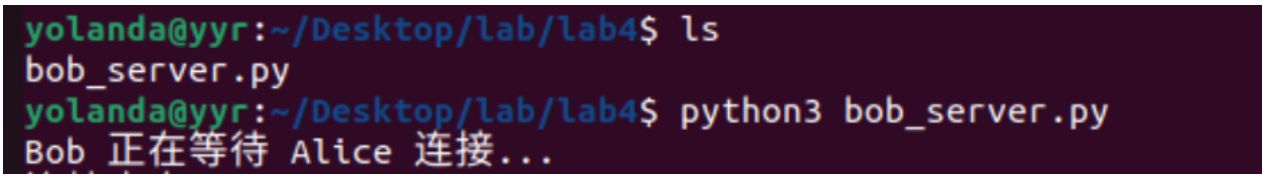
            # 验证 HMAC
            h = hmac.HMAC(k_hmac, hashes.SHA256())
            h.update(M)
            try:
                h.verify(received_hmac)
                print(" HMAC 验证成功：明文完整性校验通过。")
                print(f" 最终解密消息: {M.decode('utf-8')}")
            except Exception:
                print(" 错误：HMAC 验证失败，明文可能被篡改！")

        except Exception as e:
            print(f"验证或解密失败: {e}")

    finally:
        conn.close()
        server.close()

if __name__ == "__main__":
    bob_server()
```

先在虚拟机中运行服务端：




Alice 客户端代码如下：

```python
import socket
from cryptography.hazmat.primitives import hashes, serialization,hmac
from cryptography.hazmat.primitives.asymmetric import x25519, ed25519
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
import os

def alice_client():
    # 1. 初始化 Alice 的密钥系统
    alice_sk_x = x25519.X25519PrivateKey.generate()
    alice_pk_x = alice_sk_x.public_key()
    alice_sk_ed = ed25519.Ed25519PrivateKey.generate()
    alice_pk_ed = alice_sk_ed.public_key()

    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # Bob的 IP 地址
    bob_ip = 'lab'
    client.connect((bob_ip, 9999))

    # 2. 交换公钥 (发送 Alice 的公钥)
    client.send(alice_pk_x.public_bytes(serialization.Encoding.Raw, serialization.PublicFormat.Raw))
    client.send(alice_pk_ed.public_bytes(serialization.Encoding.Raw, serialization.PublicFormat.Raw))

    # 接收 Bob 的公钥
    bob_pk_x_bytes = client.recv(1024)
    bob_pk_ed_bytes = client.recv(1024)

    # Diffie-Hellman 密钥协商
    bob_pk_x = x25519.X25519PublicKey.from_public_bytes(bob_pk_x_bytes)
    shared_key = alice_sk_x.exchange(bob_pk_x)

    # 使用 HKDF 派生出两个密钥：k(用于加密) 和 k'(用于HMAC)
    derived_key = HKDF(
        algorithm=hashes.SHA256(),
        length=64,
```

```
        salt=None,
        info=b'handshake data'
    ).derive(shared_key)
    k_enc = derived_key[:32]     # 对称加密密钥 k
    k_hmac = derived_key[32:]    # HMAC 密钥 k'
    print(f"协商成功! \nAES密钥 k: {k_enc.hex()[:10]}...\nHMAC密钥 k': {k_hmac.hex()[:10]}...")

    # 准备数据 M 并计算 HMAC
    M = b"Hello Bob, this is a secure message from Alice!"
    h = hmac.HMAC(k_hmac, hashes.SHA256())
    h.update(M)
    hmac_tag = h.finalize()  # 得到 HMACk'(M)

    # 对称加密 C = Ek(M + HMAC)
    iv = os.urandom(16)
    payload = M + hmac_tag # 拼接明文和HMAC
    cipher = Cipher(algorithms.AES(k_enc), modes.CFB(iv))
    encryptor = cipher.encryptor()
    C = encryptor.update(payload) + encryptor.finalize()

    # Alice 对 C 签名得到 Sig
    sig = alice_sk_ed.sign(C)

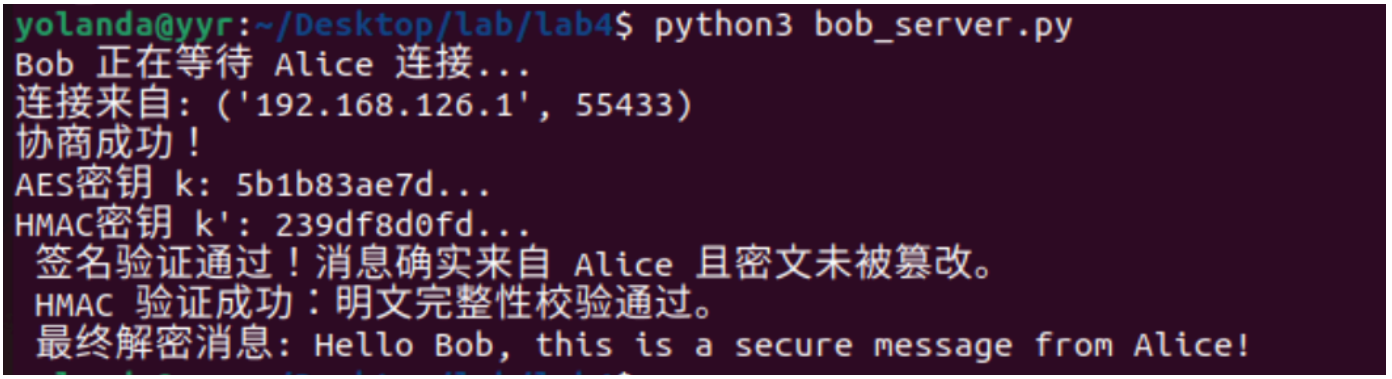
    # Alice 发送 (IV + C + Sig) 给 Bob
    # 将 IV, 密文, 签名拼接发送
    packet = iv + C + sig
    client.send(packet)
    print("加密数据和签名已发送。")

    client.close()

if __name__ == "__main__":
    alice_client()
```

运行客户端，结果如下：

```
PS D:\yyr\study\aa 专业课\网络安全技术> python -u "d:\yyr\study\aa 专业课\网络安全技术\lab4\alice_client.py"
协商成功!
AES密钥 k: 5b1b83ae7d...
HMAC密钥 k': 239df8d0fd...
加密数据和签名已发送。
```




两边密钥完全一致，最终解密后的内容与初始明文内容一致。

## （二）手搓 RSA 算法

1. **RSA算法原理**

   RSA 是一种非对称加密算法，其安全性基于**大整数分解的困难性**。其核心数学流程如下：

   - **选择两个大素数** p 和 q。
   - **计算模数** `n=p×q`。这个 n 的长度（位数）就是密钥长度。
   - **计算欧拉函数** `ϕ(n)=(p−1)×(q−1)`。
   - **选择公钥指数** `e`：要求 `1<e<ϕ(n)`，且 e 与 `ϕ(n)` 互质（通常取 `65537`）。
   - **计算私钥指数** `d`：计算 e 对`ϕ(n)` 的**模反元素**，满足 `d×e≡1(modϕ(n))`。
   - **加密过程**：$C=M^e(\mathrm{mod}n)$。
   - **解密过程**：$M=C^d(\mathrm{mod}n)$。

2. **实现逻辑**

   - **大素数检测**：采用 Miller-Rabin 概率素性检测算法，通过多次随机选取基数，确保生成的 p,q 为极大概率的素数。
   - **密钥计算**：通过 `n=p×q` 确定加密空间。利用扩展欧几里得算法求解线性同余方程 `ed≡1(modϕ(n))`，从而获得私钥 `d`。
   - **幂模运算优化**：直接计算 $M^{65537}$ 会产生极大的整数导致计算崩溃，本实验利用 Python 内置的 `pow(base, exp, mod)` 函数，其底层采用“平方-乘”算法（Binary Exponentiation），将复杂度降至 `O(loge)`。
   - **文本编码转换**：实现 `string -> bytes -> int -> RSA运算 -> int -> bytes -> string` 的完整路径。

### 3. 代码实现

```python
import random

# 扩展欧几里得算法，用于计算模逆 d
# 寻找满足 ax + by = gcd(a, b) 的整数 x 和 y
def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    else:
        # 递归调用，利用辗转相除法的原理
        g, y, x = extended_gcd(b % a, a)
        return g, x - (b // a) * y, y

# 计算模逆 公式: e * d ≡ 1 (mod phi)
# 已知公钥 e 和欧拉函数 phi，计算出私钥 d。
def mod_inverse(e, phi):
    g, x, y = extended_gcd(e, phi)
    # 如果 g (最大公约数) 不等于 1，说明 e 和 phi 不互质，模逆不存在
    if g != 1:
        raise Exception('模逆不存在')
    else:
        # x 可能为负数，通过 % phi 将其转为正整数
        return x % phi

# 素性检验 (用于生成大素数)
# n: 要检测的数
# k: 检测轮数，轮数越多，是素数的概率越接近 100%
def is_prime(n, k=5): # Miller-Rabin 测试
    if n <= 1: return False
    if n <= 3: return True
    if n % 2 == 0: return False      # 排除偶数

    # 找到 r, d 使得 n-1 = 2^r * d
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2
    # 进行 k 轮随机测试
    for _ in range(k):
        a = random.randint(2, n - 2)     # 随机选取一个证据基数
        x = pow(a, d, n)     # 计算 a^d mod n
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)     # 连续平方
            if x == n - 1:
```

```
                break
        else:
            return False     # 确定是合数
    return True      # 极大概率是素数

# 生成一个指定位数的随机大素数
def generate_large_prime(keysize=1024):
    while True:
        num = random.getrandbits(keysize)
        num |= (1 << keysize - 1) | 1  # 确保最高位为 1（保证位数）且最低位为 1（保证是奇数）
        if is_prime(num):
            return num

# RSA 密钥生成
def generate_keypair(keysize=1024):
    # 1. 选两个大素数 p, q
    p = generate_large_prime(keysize // 2)  # 每个素数占用一半的位数
    q = generate_large_prime(keysize // 2)  # 2048位密钥需要两个 1024位的素数

    # 2. 计算 n = p * q 和 欧拉函数 φ(n) = (p-1)(q-1)
    n = p * q
    phi = (p - 1) * (q - 1)

    # 3. 选择 e (通常固定为 65537)
    e = 65537

    # 4. 计算 d (e 关于 phi 的模逆)
    d = mod_inverse(e, phi)

    # 返回 ((公钥e, 模数n), (私钥d, 模数n))
    return ((e, n), (d, n))

# 加密公式：C = M^e mod n
def encrypt(public_key, plaintext):
    e, n = public_key
    # 将字符串明文编码为字节，再转换为一个大的整数 M
    m = int.from_bytes(plaintext.encode('utf-8'), byteorder='big')

    # # RSA 要求明文数值 M 必须小于模数 n
    if m >= n:
        raise ValueError("明文太长，超过模数 n")

    # C = M^e mod n
    c = pow(m, e, n)
    return c

```

```python
# 解密公式：M = C^d mod n
def decrypt(private_key, ciphertext):
    d, n = private_key
    # M = C^d mod n
    m = pow(ciphertext, d, n)
    # 将大整数 M 转换回字节，再解码为字符串
    bytes_len = (m.bit_length() + 7) // 8
    try:
        return m.to_bytes(bytes_len, byteorder='big').decode('utf-8')
    except Exception:
        return "解密失败或编码错误"


if __name__ == "__main__":
    print("正在生成 2048 位 RSA 密钥对...")
    pub_key, priv_key = generate_keypair(2048)
    print(f"公钥 (e, n): ({pub_key[0]}, {hex(pub_key[1])[:30]}...)")

    msg = "Hello, this is a RSA test!"
    print(f"\n原始消息: {msg}")

    # 加密
    c = encrypt(pub_key, msg)
    print(f"加密后的密文: {hex(c)[:50]}...")

    # 解密
    m_decrypted = decrypt(priv_key, c)
    print(f"解密后的明文: {m_decrypted}")

    if msg == m_decrypted:
        print("\n结论：加解密成功，结果一致！")
```

运行结果如下：

```
正在生成 2048 位 RSA 密钥对...
公钥 (e, n): (65537, 0xc0ed2e90be37e5396a9548eb079f...)

原始消息: Hello, this is a RSA test!
加密后的密文: 0xb864fcfb78abe7d080ba9453838660730888e1d13987bd32...
解密后的明文: Hello, this is a RSA test!

结论：加解密成功，结果一致！
```

## 五、体会与心得

通过本次网络安全技术的两次实验，我完成了从“密码协议应用”到“算法底层实现”的全过程学习，收获颇丰。

在**作业一**中，我构建了一个完整的加密通信模型。我深刻体会到，网络安全并非单一算法的堆砌，而是多种密码原语的有机结合。通过 Socket 在物理机与虚拟机间实现 X25519 密钥协商、AES 对称加密以及 HMAC 和 Ed25519 签名，我理解了“机密性、完整性、身份验证”是如何在实际协议中层层嵌套、环环相扣的。特别是“先 HMAC 再加密”与“对密文签名”的逻辑，让我对工业级安全通信的严谨性有了直观认识。

在**作业二**中，“手搓” RSA 算法则体会到了网安算法和数学之间的关联。通过编写 Miller-Rabin 素性检测和扩展欧几里得算法，我意识到非对称加密其实建立在数学原理之上。在实现大数幂模运算时，我体会到了计算效率优化的重要性——若无快速幂取模，大数运算将难以逾越内存与时间的鸿沟。